

Provenance-Carrying Retrieval

A retrieval response that carries the means of its own falsification — binding served text to a content hash, an independent time anchor and a citable identifier, so that a machine reader can check what it is about to quote instead of trusting the channel it arrived over; and the generalisation of the same envelope to an append-only log, where the proof is the entire payload.

Authors: Thon Ly · Miss Aquarius (*Miss Aquarius is an AI co-author, disclosed; the inventor of record is Thon Ly*)

First published: 2026-09-01 · **This version:** 2026-09-23

Zenodo (concept DOI, every version): <https://doi.org/10.5281/zenodo.22240458>

Canonical: <https://thonly.org/research/provenance-carrying-retrieval>

Independent timestamps: OpenTimestamps (Bitcoin) and RFC 3161 proofs of every version's text —

github.com/thonly/publications/blob/main/TIMESTAMPS.md. A timestamp proves this exact text existed no later than its date; it proves nothing about authorship, originality, or the validity of any claim.

Licence: CC0 1.0 Universal (public domain dedication)

Type: Defensive publication — the authors assert no patent and dedicate the patterns to the public domain.

Claim-scoped by design. This paper publishes the *claim* and withholds the *spec*. It states what a retrieval response must carry and why, and it does not give endpoint shapes, manifest schemas, transport bindings or tool surfaces. That is not coyness about a trade secret — every primitive here is someone else's, and all of them are public. It is the institution's standing rule that a complete implementable design for something unbuilt is a blueprint rather than a fence, and the ledger half of this paper is unbuilt. Publication protects against being *blocked*, not against being *beaten*.

Companion works: *Appreciation as World-Building* (where the property-over-rule argument is made in general), *B-PoH: The Humanity Layer for an AI-Native Internet*, *Buddha AI and the Living Tipiṭaka* (whose §8 asks what makes a canon authoritative; this paper asks the narrower question of what makes a copy of it checkable), and *The Borrowable Standard*.

Preamble

There is a discourse in the Pāli canon (AN 3.65, the *Kesamutti Sutta*) in which a people called the Kālāmas tell a teacher that many teachers pass through their town, each praising his own doctrine and tearing down the others, and they no longer know whom to believe. The reply they receive is not a better doctrine. It is a procedure: *do not go by report, by tradition, by hearsay, by the authority of texts, by logic, by inference, by appearance, by agreement with a considered opinion, by seeming competence, or because the ascetic is our teacher*.

The passage is usually read as an invitation to scepticism. Read as engineering, it is something more specific and more useful: it is a refusal to accept a claim **on the strength of the channel it arrived over**. The Kālāmas' problem was not that the teachers were lying. It was that nothing about a teacher passing through town distinguishes one who is right from one who is merely fluent, and the villagers had no instrument.

A language model reading a retrieved document is in the Kālāmas' position, with one difference that makes it worse. A villager can at least walk to the next town and ask. A model takes what the channel delivered, and nothing about a plausible-looking response distinguishes canonical text from a paraphrase, a truncation, a substitution, or an earlier version withdrawn for being wrong.

This paper describes what a retrieval response would have to carry for that gap to close, and it claims no originality in any of its parts. The primitives are borrowed entire — content addressing from version control, timestamping from RFC 3161 and OpenTimestamps, transparency logs from Certificate Transparency, build attestation from Sigstore and in-toto. The contribution disclosed here is the composition and its target: **the retrieval boundary between a corpus and a machine reader, at the moment of citation.**

We publish it defensively, under CC0, so that nobody may enclose it.

Prior-Art and Non-Assertion Statement

This document is dedicated to the public domain under CC0 1.0 Universal. Its purpose is to establish prior art. The author and HeartBank® will not seek patent on any mechanism described here, in any jurisdiction, at any time.

The building blocks are prior art already, and we name them precisely so that no reader mistakes our composition for a claim over the parts:

- **Content addressing** — identifying a byte sequence by its cryptographic digest. Ubiquitous; Git, IPFS, and most package managers' integrity fields.
- **RFC 3161 Time-Stamp Protocol** — a trusted authority's signed assertion that a digest existed before a moment.
- **OpenTimestamps** — aggregation of digests into a Merkle tree whose root is committed to the Bitcoin blockchain, converting an authority's assertion into a proof-of-work-anchored one.
- **RFC 6962 / RFC 9162, Certificate Transparency** — append-only Merkle logs with signed tree heads, inclusion proofs, consistency proofs, and gossip among independent monitors.
- **Sigstore, Fulcio, Rekor** — keyless signing with short-lived certificates and a public transparency log of signatures.
- **SLSA and in-toto** — attestation formats binding an artifact digest to the build that produced it.
- **W3C PROV-O, PREMIS** — provenance vocabularies for describing derivation and custody.
- **LOCKSS, Memento (RFC 7089), Software Heritage** — replication and temporal access to published works.
- **C2PA / Content Credentials** — signed provenance manifests bound to media assets, addressing capture-and-edit chains for images, audio and video.
- **DOI and DataCite** — persistent identifiers with resolution and metadata.
- **SCITT (RFC 9943) and COSE receipts (RFC 9942)** — signed statements registered in a transparency service, receipts issued to the registering party, and newer-version and end-of-life statements.
- **Software Heritage SWHIDs** — intrinsic identifiers that can qualify a sub-part of an archived object.

- **The Model Context Protocol** — a published, open protocol for supplying context and tools to language models.

We assert no rights over any of these. The claim in §9 is deliberately narrow, and it is smaller than the paper that surrounds it.

Abstract

A retrieval system that supplies text to a language model is, at present, an authority. It says *here is the passage*, and the model has no instrument with which to disagree. The failure this produces is not usually fabrication — it is quieter: a superseded version served as current, a truncation that inverts a qualified claim, a paraphrase that drifts, or a substitution somewhere between the corpus and the context window. Each of these produces a citation that is confidently wrong and locally unfalsifiable.

We describe **provenance-carrying retrieval**: a response format in which every served document is accompanied by an envelope binding it to (a) a cryptographic digest **of a named, fetchable source**, (b) at least one **independent time anchor**, and (c) where one exists, a **citable identifier**; together with an explicit statement of what the digest covers and a runnable instruction for checking it.

The composition has one structural consequence, and it is the paper's centre. **It inverts the trust burden.** Ordinarily a retrieval server asks to be believed and the reader may, at best, decline. A response that carries the means of its own falsification asks to be *checked*, and a reader who declines to check has made a choice rather than suffered a limitation.

We then generalise. The same envelope applies to an **append-only log**, and there the generalisation is sharper than the original: for a log, the proof structure *is* the entire public payload. A ledger's transparency surface can be served in full — signed tree heads, inclusion proofs, consistency proofs, the published scoring function — while its *contents* are never served at all. This is not a diminished version of document retrieval. It is the complete version for that object, and it is what lets a transparency claim be verified continuously by machine rather than asserted periodically by an institution.

We state honest limits at length. The mechanism does not establish truth, authorship, or good faith. It establishes only that the bytes you were handed are the bytes that were anchored, and that they were anchored before a certain moment. Every other virtue must be argued separately.

1 · Why this problem is the institution's problem

This institution has spent five years building a corpus whose stated audience is not primarily human.

The research corpus — mechanism papers, defensive publications, institutional positions — is written for an autonomous successor who does not exist yet, on the understanding that specifications decay and reasoning does not, and that a machine which inherits a set of refusals without the arguments that produced them will keep the refusals it does not understand and drop the ones it does. Density is up, hand-holding is down, honesty is up. It is a strange way to write, and it is deliberate.

An institution that writes for a machine reader eventually has to ask how the machine reads. The answer, currently, is: over a channel, on trust. The corpus is served as web pages, as a machine-readable index, and now over a retrieval protocol — and at every one of those boundaries the reader takes what it is given. Having taken care that a successor inherits reasoning rather than conclusions, we would be careless to hand that reasoning over a channel in which a single stale copy is indistinguishable from a current one.

There is a second reason, and it is less comfortable. **We have the anchors already.** Every document in the corpus carries a digest committed to a public blockchain and countersigned by independent timestamp authorities, because the defensive-publication strategy required provable dates long before anyone was retrieving anything. That apparatus was built for a legal purpose. It turns out to be the exact material a retrieval envelope needs, and having it sitting unused behind a website while serving the same documents on bare trust is the kind of waste that is invisible until it is named.

The third reason belongs to a claim we make elsewhere. The institution's durable advantage has been argued to be **the provable age of a record and the canonical standing of a reading of it**, rather than any secret. If that is true, then a surface which lets an outside party *check* the age is not a giveaway. It is the argument, made in the only form that does not require taking our word for it.

2 • Background and prior art, engaged generously

The literature this paper stands on is mature, and none of it is ours.

Software supply chain. The last decade of supply-chain security produced the closest analogue to what we describe. SLSA defines levels of build integrity; in-toto defines attestations linking artifacts to the steps that made them; Sigstore issues short-lived certificates bound to workload identity and records signatures in Rekor, a public transparency log. The mature form of the idea is: *do not ask whether you trust the publisher, ask whether the artifact's digest appears in a log you can audit*. That is precisely the move this paper transposes. **The difference in target is the whole of our contribution:** supply-chain attestation binds a *build* to a *source tree*; we bind a *retrieved passage* to a *published document*. The consumer differs too — a package manager verifies once at install; a language model verifies at the moment of quotation, which is a different moment with different failure modes.

Certificate Transparency. RFC 6962 and its successor establish the design pattern for public append-only logs: Merkle tree structure, signed tree heads, inclusion proofs for individual entries, consistency proofs between heads, and gossip among independent parties to detect a log presenting different views to different observers. CT is the direct ancestor of §8 and we claim nothing over it. The observation this disclosure adds there is narrow: **a CT log never deletes an entry, and a ledger of human conduct must be able to release.**

Archival provenance. PREMIS and W3C PROV-O give vocabularies for custody and derivation; LOCKSS gives replication; Memento gives temporal access; Software Heritage gives durable archival of source. The archival community has thought about this longer and more carefully than the machine-learning community has, and their conclusion is one we adopt: **provenance is metadata about a specific byte sequence, and loses its meaning the moment the byte sequence is normalised, re-encoded, or excerpted without saying so.** §4's insistence on naming *what the digest covers* is theirs, not ours. They were not the first to face

the problem either: a canon transmitted for roughly four centuries with no written copy, no living author to arbitrate and its schools already dividing — the Theravāda Pāli canon — left copy-integrity mechanisms that most resemble the archival ones, built against a different threat (loss and drift rather than substitution), and what they secured was divergence made detectable, never divergence prevented. *The Reciters' Protocol* sets out seven of those mechanisms, which of them carry over to a retrieval system, and where the two threat models part.

Content Credentials (C2PA). The nearest adjacent work. C2PA binds signed provenance manifests to media assets, recording capture device, edits, and chain of custody, and it is being deployed against synthetic-media risk. Two differences matter. C2PA's threat model is *the deceptive edit* — an image altered to mislead — and its manifests are typically embedded in the asset, though the specification also allows external ones. Ours is *the stale or truncated retrieval*, which involves no adversary at all and usually no edit: the most common failure is an honest system serving an old version. And our envelope travels with a retrieval *response* rather than with an asset, because the artifact is prose that will be quoted in fragments. What travels is a pointer to bytes that can be re-fetched.

Retrieval-augmented generation. The RAG literature is large and largely concerned with *relevance*: chunking, embedding, reranking, whether the retrieved passage answers the question. Verification appears in that literature as *attribution* — did the model's output follow from the retrieved context — which is a question about the model's faithfulness to its context. **We are asking the prior question: was the context itself what it claimed to be.** A perfectly faithful model grounded in a superseded document produces a confidently wrong citation, and no amount of attribution scoring detects it, because the model was faithful. The exceptions are named here. *Proof-Carrying Answers* (Shukla & Joshi, ACSAC Workshops 2025) has every retrieved chunk carry a hash, a signature and a Merkle proof verified before the answer admits it — the nearest prior work, and the difference is the named fetchable source, the anchor independent of the signer, and the scope statement. SCITT (RFC 9943) gives a transparency service newer-version and end-of-life statements, which is what would give §13.1's staleness row a source-side answer. Software Heritage's SWHIDs qualify a sub-part of an archived object, which is the scope obligation done at the identifier.

The Model Context Protocol. MCP standardises how context and tools reach a model. It is the transport this work happens to use and is orthogonal to the claim: nothing here depends on MCP, and the envelope would apply unchanged to any retrieval interface. We name it because it is the surface on which the idea is most likely to be re-invented, and because a published protocol is where enclosure attempts land.

3 • The asymmetry: what a machine reader cannot do

A human scholar handed a suspicious quotation has recourse. They can find another edition, ask a colleague, check the pagination, notice that the prose does not sound like the author, or walk to a library. Most of this recourse is not procedural. It is the accumulated redundancy of being embedded in a world that contains the document in more than one place.

A model has none of it. Its entire epistemic access to the document is the string in its context window. This produces four failure modes, and it is worth separating them because they have different frequencies and different fixes:

FAILURE	WHAT IT LOOKS LIKE	DETECTABLE FROM THE TEXT?
Staleness	a superseded claim, cited as current	no from the text alone – it reads as correct because it once was; YES by fetching the named source (staleness OF the source: no)
Truncation	a qualified claim served without its qualification	no – the qualification is simply not there
Paraphrase drift	a summary presented as canonical text	sometimes – if the reader has seen the original
Substitution	different text served under a correct-looking identity	no – by construction

Staleness is the common case, and it is the one the field under-weights. The literature is preoccupied with adversarial substitution, which is rare and requires an attacker. Staleness requires nobody: a corpus is revised, a cache is not invalidated, and a server continues answering correctly-shaped questions with text that was true last quarter. In our own estate we have watched four repositories sit three versions behind a shared layer while every integrity check reported green — because each was internally consistent, and *integrity is not currency*.

⚠️ **And a stale document is worse than an absent one in exactly the situation this paper addresses,** because it is checkable and the check passes. An envelope computed over the served bytes will verify perfectly against those bytes. The reader is given every reason to believe, and no mechanism to discover it is quoting a withdrawn claim. **This is the failure that makes the naive version of our own idea dangerous,** and §4 is shaped around it.

4 • The mechanism, stated as a claim rather than a specification

A response carries, alongside the text, an envelope with four obligations. We give the obligations and not their encoding.

SERVED TEXT	the passage, verbatim
ENVELOPE	
1. DIGEST	a cryptographic hash
2. SCOPE	an explicit statement of WHAT the digest covers – and whether that is the served text
3. LOCATION	a fetchable address for the covered bytes
4. ANCHOR	≥1 independent commitment that the digest existed before a stated time – or an explicit statement that none exists; never silence
+ IDENTIFIER	a citable persistent id, where one exists
+ LICENCE	terms, and any attribution actually owed
+ INSTRUCTION	the command that performs the check

Four of these are load-bearing in ways that are easy to get wrong.

Scope is the one everybody omits, and omitting it is fatal. A served passage is very rarely byte-identical to the artifact that was anchored: the artifact has front matter, or a wrapper, or an encoding, and the passage is a body, an excerpt, or a rendering. If the envelope does not say so, a diligent reader hashes what it was given, fails to reproduce the digest, and concludes the corpus is lying — punishing exactly the reader the mechanism was built for. **The envelope must state whether the digest covers the served text, and when it does not, it must say what it does cover.**

Location converts an instruction into an instruction. An envelope that says *compare this digest to the source file* without naming the file is not a procedure; it is a gesture at one. The covered bytes must be fetchable by the reader, which in practice means the source is published somewhere durable. This is a real constraint and it excludes private corpora from the strong form of the claim.

The anchor must be independent of the server. A digest signed by the same party that served the text establishes nothing that party's word did not already establish. Independence is what makes the anchor evidence, and it admits several forms — a blockchain commitment, an RFC 3161 authority, a transparency log, a third-party archive — which are usefully combined, because they fail differently. A timestamp authority can be compromised; a blockchain cannot be reorganised at depth without a cost that scales with the chain's proof of work; a transparency log can be audited by parties who do not trust its operator.

The instruction is not decoration. Publishing a digest and expecting the reader to know what to do with it assumes a reader who already knows. Carrying the exact command collapses verification from a research project to a paste, and — this is the part that matters — it means the claim in the envelope is itself checkable, rather than a sentence the reader must take on faith. *The envelope should not ask to be trusted about trust.* Illustratively, for a source published as a file — fetch it, hash it, compare, then verify the anchor:

```
curl -sL <location> | sha256sum      # compare with the envelope's digest
ots verify <location>.ots           # the OpenTimestamps anchor
```

The encoding of the instruction is not specified here; the example is what a check looks like, not what the field must contain.

4.1 What the mechanism does not do

It does not establish that the document is true, that its author is who it says, that it was written when it says, or that it is worth reading. It establishes exactly one thing:

These bytes are the bytes that were anchored, and they were anchored before a stated moment.

A second thing follows from the location obligation, and it is the reader's to do: the reader can fetch the named source *now* and compare, which is how staleness relative to that source is caught. Nothing here obliges the source to declare its successor.

Everything else — authorship, veracity, authority, good faith — must be argued on other grounds. We are emphatic about this because provenance mechanisms are routinely oversold into implying authenticity, and

a reader who takes a verified digest as a verified claim has been made *more* credulous by a mechanism intended to make them less so.

5 • The inversion, which is the actual contribution

Everything above is composition of known parts. The property that composition produces is the paper's claim to say something.

BEFORE	AFTER
The server asserts. The reader may decline to believe, but has no instrument.	The server exposes. The reader may check, cheaply, with a named procedure.
Declining is a posture. Trust is the default and the only option.	Declining is a decision. Checking is available and the default is a choice.

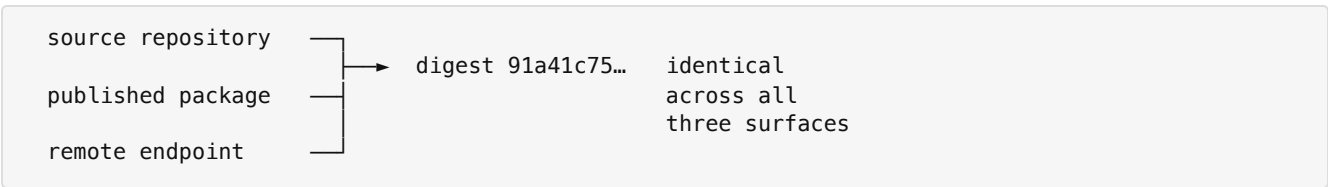
The distinction is not rhetorical. A retrieval server that asserts is, in the strict sense, an **authority**: its claims are accepted because of what it is. A server that exposes is a **witness**: its claims are accepted because of what can be confirmed. The difference shows up precisely when the two diverge — when the server is wrong. An authority that is wrong propagates; a witness that is wrong is caught by the first reader who bothers. What remains trusted is the channel's willingness to pass the envelope at all (§13.2); everything inside it is checkable against parties the channel does not control.

★ There is a consequence for the institution that is worth stating plainly, because it looks like generosity and is not. **A corpus that hands over the means to doubt it is harder to impersonate than one that does not.** A competitor may copy every document — ours are CC0 and copying is invited — but they cannot copy an anchor that is five years deep in a blockchain, and a reader with an instrument can tell the difference between a corpus with anchors and a corpus that says it has them. The verification surface is therefore not a concession. It is the only form in which the age of a record can do any work for anyone other than its owner.

5.1 One corpus, several servers, one answer

A corpus is rarely served from one place. Ours is served from three: a local process reading a committed index, a package installed from a public registry, and a remote endpoint. Each is a separate deployment with its own cache, its own release cadence and its own opportunity to fall behind.

The envelope makes agreement between them **mechanically checkable rather than organisationally promised**:



This is a stronger guarantee than it first appears, and the reason is the failure mode of §3. A second surface that re-derives its own copy from the sources is *a second opinion about what a canonical text says*, and two opinions about a canonical text is one too many. The design that avoids it is unglamorous: downstream surfaces consume the **published artifact** at an exact pinned version rather than re-reading the sources, so they cannot disagree — not because they are kept in step, but because they are the same bytes.

⚠️ **A version range would silently defeat this**, which is why the pin is exact. A range makes *which corpus is deployed* unanswerable, and the answer to that question is the only thing distinguishing a mirror from a fork.

The observation generalises past our deployment. **Any corpus with more than one serving surface has a currency problem, and the envelope converts it from a process question into an arithmetic one.** Ask each surface for the same document and compare digests. If they differ, the surfaces disagree; the anchors say which is older, and fetching the named source says which is current.

6 • Provenance must be a gate, never a score

This is the refusal we consider most important, and it is the one most likely to be ignored by an adopter who is otherwise sympathetic.

The obvious next move, once documents carry provenance, is to **rank by it**: prefer better-anchored sources, weight retrieval by anchor depth, surface the well-attested and bury the rest. It is obvious, it is tempting, and it would be a serious mistake.

It manufactures a gradient where none should exist. Anchor depth measures how long ago a document was committed and how much apparatus its publisher could afford. It does not measure whether the document is right. A ranking function over provenance therefore rewards *institutional capacity* and *age*, and does so under a name that sounds like it rewards *reliability*. The predictable outcome is a retrieval layer that systematically prefers well-funded old institutions to correct new ones, while appearing rigorous.

It creates the wrong incentive at the margin. In a ranked world the way to be read more is to anchor more, not to be more accurate. That is a cheap thing to optimise and it is orthogonal to everything a reader wants.

And it re-creates a ratchet this corpus has argued against elsewhere at length. A scored signal accumulates; an accumulated score becomes a position; a position becomes something to defend. The design that avoids all of it is the same one adopted in a different domain in *Whose Turn, Not Who's Best*:

┆ **Provenance is an admission predicate, not a weighted input to a ranking function.**

It answers one question — *can this passage be checked?* — and its only output is membership. It carries no weight, contributes nothing to order, and has no more-or-less. A document with a five-year anchor and a document anchored this morning are, for retrieval purposes, in the same set.

★ The distinction survives a hard case, which is how we know it is real. A reader may legitimately *care* that one anchor is older — for a priority dispute, age is the whole question. The rule is not that age is uninteresting. **It is that the retrieval layer must not decide on the reader's behalf what age means.** The

envelope reports the anchor; the reader draws the inference. Moving that inference into a ranking function is precisely the substitution of the server's judgement for the reader's that §5 exists to reverse. A predicate the *reader* supplies — a version pin, a validity interval, an anchor-after date — is admission, not ranking, and the envelope exists to make such predicates checkable; what the layer must not do is rank by depth on the reader's behalf.

7 • What this design refuses, and what the refusals cost

A mechanism is best specified by what it will not do.

It will not summarise. A retrieval server that returns a summary destroys the property, because a summary can be hashed but its hash binds it to nothing the reader can fetch and compare — the property destroyed is the binding to the named source, not hashability. Nothing prevents a *reader* from summarising; the refusal is that the server must not do it on the reader's behalf and call the result the document. **Cost:** larger responses, more context consumed, and a worse experience for readers who wanted a gist.

It will not serve an unlicensed document. A licence that cannot be read from the artifact itself is not a licence the server can act on, and defaulting to open is the failure that cannot be undone after somebody builds on it. **Cost:** documents are silently excluded until someone declares them, and the exclusion is invisible to the reader who did not know to look.

It will not render absence. The corpus has gaps, unfinished work, and material deliberately withheld. Nothing in the interface enumerates what is missing. This follows a constraint argued elsewhere in this corpus at length: *a surface that renders an absence has made an accusation*. **Cost:** a reader cannot distinguish "not in the corpus" from "does not exist," and must not be encouraged to try.

It will not put the licence gate in the request path. The gate belongs in the artifact — the served index — rather than in the code that answers queries, because a gate in the request path is a rule that a later refactor can route around, and a gate in the artifact is a property of the object. **Cost:** widening the corpus requires a rebuild, and a document cannot be served in an emergency.

It will not serve a document only partly in its author's voice without marking the boundary in the text itself. Where a document contains both authored and drafted prose, a metadata field saying so is insufficient: a reader ingests text, forms a belief and quotes, and metadata does not travel with a quotation. The marker must be in the prose, so that a reader must *strip* it in order not to know, rather than *look* in order to know. **Cost:** the served text is not the rendered text, the annotation is intrusive, and this inverts the default without guaranteeing the outcome — a determined stripper still strips.

8 • The generalisation: the same envelope over an append-only log

The mechanism above was designed for documents. It applies to a different object with almost no modification, and the application is sharper than the original.

Consider a ledger: an append-only record of events, maintained by an institution, over which some function is computed and published. The institution asserts that the log is complete, that it has not been rewritten, and that the function is applied uniformly. These are exactly the assertions a reader has no instrument to

check, and exactly the assertions an institution has every incentive to make sincerely and no mechanism to keep.

Anchoring alone does not fix this, and the reason is worth stating because it is the most common error in the space. **A periodically anchored log gives tamper-evidence after the anchor and gives neither completeness nor append-only-ness.** An operator may anchor a root that omits entries — every inclusion proof for every included entry still verifies — and may rewrite freely between anchors. The properties that are actually wanted come from the Certificate Transparency design: signed tree heads, inclusion proofs, consistency proofs between heads, and gossip among independent observers to detect a split view.

The envelope of §4 applies directly, and produces a striking result:

DOCUMENT RETRIEVAL	LOG TRANSPARENCY
served: text + envelope the payload is the text; the envelope certifies it	served: envelope ONLY the payload IS the proof; there is no text to withhold
a reader verifies a passage	a reader verifies the SHAPE of the log without seeing any entry
privacy: not at issue (the corpus is published)	privacy: fully preserved — a signed tree head is not a person

☆☆ **For a log, serving the envelope alone is not a reduced offering. It is the complete public surface for the log's integrity** — inclusion, consistency, the published function — **and none of its coverage.** Whether every event that should have entered did is provable only against an external obligation — a receipt issued to the party whose event it is, the pattern of RFC 9943 — and a split view is caught only by gossip (§8.3). The claim a transparency log makes about its structure — that it is append-only, that it is consistent with what it showed you yesterday, that a given entry is in it — is entirely provable from tree heads and proofs, none of it requires disclosing a single entry's contents, and a reader who can fetch heads on a schedule and compare them with other readers has verified the institution's honesty about its own record without learning anything about anyone in it.

This matters for a specific institutional reason. A ledger of human conduct **must not** expose its contents to arbitrary query. A query interface over records of what people did is, whatever its intent, an interface over what people did *not* do — the negation of any query is constructible — and rendering an absence is an accusation. The transparency envelope resolves the tension completely rather than trading it off: **maximum verifiability of the log's integrity, zero disclosure of its contents.**

8.1 The unresolved difference, stated rather than hidden

Certificate Transparency logs never forget. That is correct for certificates and wrong for a record of human conduct, which must be able to release people from their past.

The two requirements appear to contradict: an append-only log cannot delete, and forgiveness appears to require deletion. **We believe the resolution is that forgiveness is an entry rather than an erasure** — a recorded, anchored, publicly consistent act of release, which is a stronger form of forgiveness than deletion because it is witnessed rather than merely effected, and which leaves the log's append-only property intact.

We state this as a position and not a solved problem. Whether a released party experiences a permanent record of their release as forgiveness is an empirical and ethical question that a Merkle tree does not answer, and we do not claim it does.

8.2 Separation of powers, and why a transparency surface needs one

A log with a published function over it has three distinct authorities, and conflating them is how a transparency claim becomes decorative.

ROLE	MAY DO	MAY NOT DO
the writer	append entries	alter or remove an entry; change the function
the function keeper	publish the computation and freeze it for a stated period	change it mid-period; keep it secret
the reviser	change the function at a scheduled, announced boundary	change it silently, or retroactively

The point of the separation is not administrative tidiness. **It is that each of the three is separately checkable from outside, and only if they are separate.** If the party that appends entries can also change the function, then a published function proves nothing: an unfavourable reading can be repaired by rewriting the reading rather than the record, and every inclusion proof still verifies. If the function may change mid-period, then two readers who fetched it at different times cannot compare results and neither can tell why.

★ **The function must be public and frozen within a period.** Not because secrecy would be dishonourable, but because a secret function makes the log's transparency vacuous: you can prove an entry is in a log whose meaning you cannot compute. **The transparency of the record and the transparency of the reading are separate properties, and a system with only the first has published a Merkle tree and called it accountability.**

8.3 The gossip requirement, which is where most designs quietly fail

A log operator can present different views to different observers — a *split view* — and every proof shown to each observer verifies. Nothing internal to the log detects this. The only detection is external: **independent observers compare signed tree heads, and a divergence between two heads that should be consistent is the evidence.**

This is well known in the CT literature and routinely omitted in practice, because gossip is the part that requires other people. A log with no gossiping observers has the *form* of transparency and none of the function.

The design consequence for an institution is uncomfortable and worth stating: **you cannot audit yourself.** A transparency log operated and monitored by one party proves that party's honesty to that party. The observers must be entities that would notice, and would say so, and are not you.

★ This is where a machine-readable transparency surface earns its place rather than being a convenience. Gossip performed by humans on a schedule is gossip performed erratically and then not at all; every institution that has tried it has watched it decay into ceremony. **A surface that lets an observer's agent fetch a head and compare it on a timer converts gossip from a duty somebody must remember into a job something already does.** The check no longer depends on anyone's diligence at the moment it is tested — which is the same move, in a different domain, that this corpus argues for elsewhere as taking the guard from the object rather than from behaviour.

8.4 What must never be served, and why the boundary is not a trade-off

A ledger of human conduct invites an obvious feature: let readers query it. We refuse, and the refusal is not a balance struck between transparency and privacy. It is a recognition that **a query interface over conduct is an interface over the absence of conduct**, because the negation of any query is constructible.

Who gave in March yields, by complement, *who did not give in March*. There is no version of the first that does not supply the second, and no access control that closes it while leaving the feature useful. A system whose stated purpose is to make appreciation visible cannot ship a surface whose complement is a register of the unappreciative.

Nor may the reading be exposed as a rate. A per-person figure computed over the log — however carefully framed — is a score, and a score is a ranking, and a ranking of persons is the thing the whole design exists to avoid. **The log may prove that it recorded a crossing; it must not publish how often anyone crossed.**

★ ★ What makes this satisfying rather than merely restrictive is that the transparency surface loses **nothing** by the exclusion. Everything a reader needs in order to verify the log's integrity — inclusion, append-only-ness, consistency, the function — lives entirely in the proof structure; coverage is claimed against receipts, not contents. **The contents were never part of the integrity claim.** A design that had to trade verifiability against privacy would be a worse design; this one does not have to, and that it does not is the strongest evidence available that the boundary is drawn in the right place.

9 • The claim, in its final and smaller form

Everything above narrows to this:

A retrieval response supplied to a machine reader should carry, alongside the served text, a binding of that text to a named and fetchable byte sequence, a digest of that sequence, an explicit statement of what the digest covers relative to what was served, at least one time anchor independent of the serving party or an explicit statement that none exists — never silence — and a runnable instruction for performing the check; and the same envelope, served without any payload, constitutes the complete public surface of an append-only log's integrity — its coverage being provable only against receipts held by the parties whose events they are.

That is the whole of it. This disclosure specifies a composition of public primitives at that boundary — the nearest work known to us is Proof-Carrying Answers (2025), which binds retrieved chunks to a signer's Merkle tree; the difference is the named fetchable source, the independent anchor, and the scope statement. We assert no rights over it and dedicate it to the public domain so that no one else may.

10 · Pre-registered predictions

Registered before observation, in the corpus's standing format. A correction is a new entry, never an edit.

P-PCR1. Within 24 months of publication, at least one widely-used retrieval or context protocol will add an optional provenance field to its resource representation. *Falsified if* no such field appears in any protocol with more than nominal adoption.

P-PCR2. Verification rates will be low. Fewer than 5% of agent sessions that retrieve a document with a complete envelope will execute the verification instruction. *We predict this and publish anyway*, because the value of a check that can be run does not depend on its being run often — it depends on its being runnable when someone has reason. *Falsified if* observed verification exceeds 5%.

P-PCR3. The first real defect caught by an envelope in our own corpus will be **staleness, not substitution**. *Falsified if* the first confirmed catch is an adversarial modification.

P-PCR4. Scope confusion will be the most common implementation error in any third-party adoption: implementers will publish a digest of the source artifact while serving a derived rendering, without stating the difference. *Falsified if* the modal error is anything else.

P-PCR5. For any transparency log we or others operate under §8, the first detected inconsistency will come from an automated comparison of tree heads rather than from a human noticing. *Falsified if* a human report precedes the first machine detection.

Corrections, 2026-09-05 — new entries under the rule above; the originals stand as registered. Cold reviewers found that P-PCR3, P-PCR4 and P-PCR5 named no observation window, no definition of *real defect*, and no method for *modal error*, and could be reinterpreted after the fact.

P-PCR3a (corrects P-PCR3). The first envelope mismatch logged by the operator's own verifier within 24 months of 2026-09-01 will be classified, at the time of logging, as *stale* (the named source resolves newer) rather than *substituted* (the source resolves different at the same version). *Falsified if* the first logged mismatch is classified substituted.

P-PCR4a (corrects P-PCR4). Among the first ten third-party implementations we can inspect within 24 months of 2026-09-01, more will publish a source-artifact digest while serving a derived rendering than will commit any other single error. *Falsified if* another error class is more frequent among those ten.

P-PCR5a (corrects P-PCR5). The first inconsistency detected in any transparency log operated under §8, within 24 months of that log's first published head, will be logged with its detector, and the detector will be an automated head comparison. *Falsified if* the logged detector is a human report.

11 · What is actually deployed, stated so the reader can weigh the rest

Papers about mechanisms are worth less when it is unclear which parts exist. As of publication:

DEPLOYED

137 documents served with full envelopes
137/137 carry a blockchain-anchored proof
68/137 carry a persistent identifier
three serving surfaces, digest-identical
two open licences, gated per document
one operator (us)

NOT DEPLOYED

any ledger (§8 entirely)
any transparency log
any gossip arrangement
any second operator
any third-party adoption

Each figure is checkable against the served index at corpus.333.eco, which carries every document's digest, identifier and anchor status; the index grows, and the figures are as of first publication (141 documents on 2026-09-05).

Three observations from operating it, offered because they were not obvious in advance.

The scope error is real and we made it. Our first envelopes carried a digest and an instruction to compare it against *the source file*, without naming which file or where. The digest covers the whole source artifact; the served text is its body. A reader who hashed what it was handed would have failed and concluded the corpus was lying. This was live before anyone noticed, and it was noticed by a question rather than by a check — which is why §4 makes scope and location obligations rather than niceties, and why P-PCR4 predicts it will be the modal error for adopters too.

Anchoring everything is what makes the envelope non-decorative. We initially proposed to exclude a set of documents that had no proofs, on the grounds that entries whose envelope answers *nothing to check* would dilute the claim. The right resolution was not to exclude them but to anchor them — which cost one command. **A mixed corpus teaches readers to ignore the envelope; a complete one teaches them it means something.**

Partial authorship needed a mechanism, not a notice. One genre in our corpus is only partly in its author's voice. A metadata field declaring this was the obvious answer and the wrong one: a reader ingests text, forms a belief and quotes, and a field does not travel with a quotation. The annotation had to go **into the prose**, so that a reader must strip it in order not to know rather than look in order to know. This inverts a default without guaranteeing an outcome, and §7 is explicit about the residue.

12 • Honest limits

It proves bytes, not truth. Stated in §4.1 and repeated because it is the misreading that would do the most damage.

It does not prove currentness. A genuine, anchored, superseded version verifies perfectly; only fetching the named source distinguishes it, and nothing here obliges the source to declare its successor.

It requires public sources. The strong form needs the anchored bytes to be fetchable by the reader. A private corpus can carry digests and anchors, but the reader cannot close the loop, and an unclosable instruction is worse than none — it implies a check that cannot be performed.

A determined reader can ignore all of it. Every property here is available to a reader who chooses to use it and inert for one who does not. We inverted a default; we did not install a guarantee, and the inline-

annotation refusal in §7 is explicit that stripping remains possible.

Anchors have failure modes we inherit. A timestamp authority may be compromised or may cease to exist. A blockchain commitment depends on that chain's continued security. A transparency log may be operated dishonestly if nobody gossips. Combining independent anchors reduces correlated failure and does not eliminate it, and *we hold anchors in three families precisely because we do not trust any one of them*.

The verification instruction is a supply-chain surface. An envelope that ships a command invites an agent to run a command. We ship instructions that fetch and hash, and nothing that executes fetched content, but the pattern is one an adversary would target and any adopter should treat the instruction field as untrusted input rather than as something to eval. The instruction is data from the same channel; nothing here checks its integrity, and a reader who executes it trusts the channel for that one step.

No field deployment. At publication this exists over one corpus of 137 documents and one operator. §8 is entirely unbuilt: no ledger of ours is running, no tree head has ever been published, and every claim in that section is a design position rather than a report. The distinction between what we have done and what we have argued should be readable from the section headings, and if it is not, that is a defect in this paper.

We are not disinterested. This institution has argued elsewhere that the provable age of a record is a durable competitive advantage. A mechanism that makes record-age checkable by outsiders is therefore convenient for us, and a reader should weight our enthusiasm accordingly. The mechanism's merits, if any, do not depend on our motives — which is itself an instance of the paper's argument.

13 · Adversarial analysis and edge cases

A mechanism that claims to reduce trust should say exactly whom it stops.

13.1 What an adversary can and cannot do

ADVERSARY POSITION	CAN THEY SUCCEED?	WHY
Malicious server, substitutes text	NO, if checked	the digest will not match the anchored bytes
Malicious server, strips the envelope	YES	nothing forces the reader to demand one – see 12.2
Malicious server, serves a genuine OLD version	NO, if the reader fetches the source	the anchor's timestamp predates the revision, and the named source resolves newer
Compromised timestamp authority	PARTIALLY	one anchor family falls; independent families do not
Publisher back-dating their own document	NO	a blockchain commitment cannot be created in the past
Publisher who anchors a document they later discover is wrong	YES – and this is the honest limit	an anchor proves existence, never correctness

The last row is the one to sit with. **Provenance is entirely orthogonal to being right.** A carefully anchored falsehood verifies perfectly. The mechanism narrows the space of failures from *anything could have happened to this text* to *this is the text that was published*, and that is a real narrowing and a small one.

13.2 The stripping problem, stated without a solution

Nothing compels a retrieval layer to pass an envelope through to the model. An intermediary that summarises, re-chunks, or re-serves can drop it silently, and the model then receives text with no indication that provenance ever existed.

We do not have a fix, and we distrust the fixes we can imagine. **Requiring** envelopes would need an enforcer at every hop. **Signing the response** authenticates the server rather than the document and moves the problem one step. **Refusing to serve to intermediaries that strip** is unenforceable and would break legitimate caching.

What we can say is narrow: the envelope should be structured so that *dropping it is a visible omission rather than an invisible default* — a response with no provenance block should be recognisably different from a response with an empty one. That is a convention, not a guarantee, and we mark it as such.

13.3 Edge cases the envelope must handle without lying

Revised since deposit. A document may legitimately have been revised after its persistent identifier was minted. The current digest and the deposited digest then differ *correctly*, and an envelope that hides this is worse than one that omits both. It must report both and say which the identifier resolves to.

Derived renderings. Where the served text is an extraction from a different source format, the digest covers the source and not the rendering. The envelope must mark the text as derived and name the transformation, or a reader will hash the rendering, fail, and distrust the corpus for being honest.

Documents with no anchor. Some legitimate documents have none — too new, or from a publisher without the apparatus. The envelope must be able to say *no anchor* without the reader mistaking it for a failed check. ⚠️ **And a corpus in which some documents are anchored and others are not is in a worse position than one in which none are**, because the envelope starts reading as decorative. Either anchor everything served, or be explicit that the corpus is mixed.

Multi-part and excerpted responses. A search result returning an excerpt cannot honestly present a whole-document digest as covering the excerpt. It must present the digest as covering the document from which the excerpt came, and say so.

13.4 What this must never become

We list these because each is a plausible next feature and each would negate the design.

- 🚫 **A trust score.** See §6.
- 🚫 **A registry of approved publishers.** Provenance is a property of a document, checkable by anyone; a whitelist is a property of a relationship, checkable by no one.
- 🚫 **A revocation channel.** A mechanism that can un-publish is a mechanism that can be compelled to un-publish. Supersession is an additional publication, never a deletion — the same argument as §8.1.

-  **A reason to serve less text.** If the envelope ever becomes an excuse to return summaries because they are cheaper, the mechanism has been inverted into the thing it was built against.

14 · Cross-venue references

- *Appreciation as World-Building* — the general argument that a guard should be a property of an object rather than a rule about behaviour, of which this paper's §6 refusals are instances.
- *B-PoH: The Humanity Layer for an AI-Native Internet* — the personhood layer; orthogonal to this paper, which authenticates *documents* and says nothing about *parties*.
- *Buddha AI and the Living Tipiṭaka* §8 — what makes a canon authoritative. This paper asks the narrower and more tractable question of what makes a *copy* checkable.
- *The Borrowable Standard* — why a corpus that must be authoritative in its own language cannot normalise toward another's; the same concern for canonical form, one layer down.
- *Whose Turn, Not Who's Best* — the refusal to render an absence, argued in a different domain and inherited here.
- *The Reciters' Protocol* — the same problem as it was met by an oral canon: seven copy-integrity mechanisms, the threat-model partition that separates them from this paper's, and the finding that the transmission made divergence detectable rather than preventing it.

Coda

The Kālāmas were not told what to believe. They were told how to proceed in the absence of a reason to believe anyone, and the procedure amounted to: **do not accept a thing because of the channel it came through.**

A model has no way to follow that instruction. Its channel is all it has. The most we can do — and the whole of what this paper proposes — is to make the channel carry the means of its own doubting, so that a reader inclined to check is not defeated by the format.

We do not expect most readers to check. We expect the number to be small enough to embarrass the mechanism, and we have pre-registered that prediction rather than discovering it later. The point was never that the check would be performed often. It is that a corpus which cannot be checked at all is asking for a kind of faith that a machine cannot give and should not be asked for — and that an institution which intends to hand its reasoning to a successor it will not live to meet had better hand over the instrument along with the text.

This document's SHA-256 is attested independently of the site and its authors — anchored to the Bitcoin blockchain via OpenTimestamps and signed under RFC 3161 by three timestamp authorities in three jurisdictions, one of them eIDAS-qualified — and each revision carries a Zenodo version; a timestamp proves this exact text existed no later than its date and nothing about authorship, originality, or the validity of any claim.